

The Dry Stack

Survival of the fittest, revisited. A working argument that optimization is good for the world.

TL;DR, don't read, click: [live market wall](#), [F1 replay](#), [break a cluster](#), [live benchmarks](#). Real engines, real queries, live right now, not screenshots.

A technical overview for engineers, architects, and anyone deciding where compute should go next.

A database, a message queue and a serializer, fused into one process and tuned to the SIMD instruction. Tens of millions of writes per second. Microsecond queries. One machine.

In one paragraph

Most systems are three separate machines wearing a trench coat: a database to store data, a message queue to move it, and a serializer to pack it for the wire. Every seam between them is a copy, a serialization pass and a network hop, overhead you pay forever, on every record. The Dry Stack fuses all three into one process and tunes the result down to the SIMD instruction. The payoff is not a rounding error: on a single machine it ingests **tens of millions of records per second** and answers queries in **microseconds**, matching or beating clustered, purpose-built systems that run on an order of magnitude more hardware. Everything in this document is running live at [drylane.io](#): real engines serving real queries.

The point isn't just that it's fast. The point is what fast *means*: the same work on far less hardware, far less energy and far less operational weight. Optimization is not a luxury or a hobby. It is the most direct lever we have on cost, on power draw, and on how good it feels to build software. This is the case for pulling that lever.

The problem: we've normalized over-provisioning

Somewhere along the way the industry decided that hardware is free and engineering time isn't, and then stopped checking whether that was still true. It quietly stopped being true years ago.

The default answer to "we have a lot of data and need it fast" is now reflexive: reach for a distributed database, put a message bus in front of it, wrap everything in containers and schedule the containers across a cluster. Each layer is reasonable on its own. Together they form a stack where *most* of the CPU cycles spent on a request go to moving bytes between components that never needed to be separate: serializing, copying, hopping, deserializing, and doing it all again on the way back.

We've learned to read the symptoms as the cost of doing business: the cloud bill that grows faster than the product, the cluster that needs a full-time team to keep upright, the "it's slow, add more nodes" reflex. But throwing nodes at a problem that is really a *design* problem doesn't fix it. It rents around it, monthly, forever, and burns the electricity to match.

This document is not a complaint about any particular tool. Databases, brokers and orchestrators are genuinely good at being general-purpose, and general-purpose is the right call for a great many teams. The argument here is narrower and more hopeful: **a huge share of that overhead is not fundamental**. It's there because nobody fused the pieces. When you do, the numbers move by factors, not percentages, and everything downstream of those numbers moves with them.

Optimization as a value, not a vanity

Three reasons this matters beyond the leaderboard.

- Economic.** If one box does the work of a cluster, the cluster's cost mostly evaporates: machines, licences and the people who babysit them. Efficiency is the only cost lever that doesn't ask you to ship less or charge more.
- Environmental.** Every unnecessary copy is an electron pushed for no reason. Data centers are a fast-growing slice of global electricity demand, and the marginal watt increasingly comes from wherever the grid can find it. Code that does the same job on a tenth of the hardware draws roughly a tenth of the power, and needs roughly a tenth of the machines built, shipped, cooled and eventually landfilled. Efficiency is climate policy you can merge.
- Experiential.** Fast is fun. Systems that answer before you've finished asking change how you think: you explore instead of wait, and you ask the expensive question because it's no longer expensive. A generation of engineers has been taught that "good enough" is the ceiling and that latency is someone else's SLA. It isn't, and it doesn't have to be. Aiming higher is not nostalgia for hand-tuned assembly. It's a bet that the world is better when the tools underneath it are excellent.
- The Dry Stack is the existence proof. Here's what's in it.

What the Dry Stack is

Three tightly integrated components in one process, no glue, plus a messaging layer built from all three.

COMPONENT	REPLACES	WHAT IT DOES
Facts	the database	Embedded time-series store. SIMD queries over billions of records, answers in microseconds.
Nio	the transport	Zero-copy async I/O and messaging. Tens of millions of msg/s on one connection, hundreds of millions fanned out.
Zao	the serializer	Generated serializers whose fast path is a raw memcpy, the speed of memory itself.
Relay	the broker	Durable pub/sub where the log is the broker: publishing is an append, replay is a scan.

The engine core is **C++**, lean and allocation-averse. On top sits an idiomatic **C#** layer with zero-copy wrappers: you write your application in C#, and the hot path pays no marshallng tax and no per-record allocation. One codebase runs on **Windows and Linux**, SIMD-tuned for **Intel, AMD and ARM**:

the same engine from a server rack down to a \$35 Raspberry Pi.

Because the pieces share one address space, the seams disappear. A message doesn't get serialized, copied to a socket buffer, sent, received, copied again and deserialized. It goes from buffer to wire and back. A stored fact doesn't cross a network to be queried; the query runs where the data already lives. Deleting the seams is where the order of magnitude comes from.

How it's actually fast

No single trick. A stack of deliberate choices that compound:

- **Fusion removes the hops.** Store, transport and serialization live in one process, so the copies and round-trips between them simply don't exist.
- **Zero-copy everywhere.** Buffers are pre-allocated and reused; data moves by reference, not by duplication. Nio's buffers are GC-free and cost the same whether you hold 64 connections or 10,000.
- **Columnar and SIMD.** Facts stores fixed-size fields column by column, so a SIMD kernel streams one column at full memory bandwidth instead of hauling whole rows through cache.
- **Blit serialization.** When a struct's memory layout already matches the wire, Zao's top tier just copies the bytes. No field walking, no encoding, no allocation.
- **Modern async I/O.** Nio sits on `io_uring` (Linux) and RIO (Windows), draining many connections with a handful of poll threads and no locks, instead of one thread per connection paying about 1 MB of stack each.
- **Saturate one box first.** The whole stack is designed to fill a single machine before it reaches for a second. Most workloads never need the second.

None of these is exotic on its own. The result is unusual because they're applied *together*, to every layer, at the same standard, so no slow component quietly caps the rest.

Proof, not slides

Every number below is a median from our benchmark harness, on real data, on a single **AMD Ryzen 9 9950X** (16 cores, 32 threads). Each one lives on a page you can check.

Facts vs DuckDB: in-process, TSBS data

Same 8.19-million-row dataset, warm. Facts ran on **one core**; DuckDB got **all 32**.

METRIC	FACTS (1 CORE)	DUCKDB (32 CORES)	GAP
Ingest	20 M/s	2.3 M/s	8.7× faster
Latest-value point query	~0.2 μs	~4.3 ms	~21,000× faster
High-CPU filter (TSBS)	107 M/s	47 M/s	2.3× faster
Full-range aggregate	1.6 ms	7.9 ms	4.9× faster
Columnar scan (SIMD)	66 GB/s	7 GB/s	9.4× faster
Storage per point	96 B	25 B	3.8× <i>more</i> (honest loss)

The one row Facts loses is storage density, and it's a deliberate trade. Facts keeps *column* data raw and memory-mapped so SIMD kernels can scan it at full memory bandwidth; DuckDB compresses. *Blob* payloads are a different story: Facts compresses them through a plug-in codec (LZ4 is built in) and decompresses transparently on read, so logs, documents and packets still pack down. Details: drylane.io/facts.html.

Facts vs QuestDB and InfluxDB: over the wire

The way you actually run those databases: client and server, every query crossing a connection. Facts used a **single connection**; the rivals got **eight each**, their fastest setup.

METRIC	FACTS (1 CONN)	QUESTDB (8)	INFLUXDB (8)
Ingest	24.6 M/s	1.65 M/s (15×)	0.24 M/s (102×)
Latest-value point query	39 μs	260 μs (6.7×)	56 ms (~1,400×)
Range read	0.25 ms	1.15 ms (4.6×)	36 ms (144×)
High-CPU filter (TSBS)	5.2 ms	105 ms (20×)	846 ms (163×)

With full durability, every engine writing to a real NVMe SSD instead of RAM, Facts held **25 M rows/s**. Most of the gap comes from the wire: Facts speaks a compact binary format, while the rivals use text line protocols that spell every number out in ASCII and parse it again. Where the rivals win: they compress at rest, and both are full query languages with far bigger ecosystems.

Nio vs the field: loopback, 64-byte messages

TRANSPORT	ROUND-TRIP	THROUGHPUT	BANDWIDTH
Nio	16 μs	28 M msg/s	2.67 GiB/s
raw sockets	52 μs	28–40 M msg/s	2.34 GiB/s
NetCoreServer	33 μs	4.0 M msg/s	0.39 GiB/s
WebSockets	67 μs	75 K msg/s	0.86 GiB/s
gRPC	179 μs	49 K msg/s	0.15 GiB/s

Against the frameworks people actually reach for, Nio's round-trip is 2 to 11 times faster. And where thread-per-connection collapses, Nio scales: **64 connections drained by 12 threads at 206 M msg/s**, about 7× the work per thread, with no locks. Details: drylane.io/nio.html.

Zao vs the field: encode into a reused buffer, nanoseconds (lower is better)

PAYLOAD	ZAO	MEMORYPACK	MESSAGEPACK	PROTOBUF	JSON
PlayerMoved (16 B blit)	0.87 ns	3.11 ns	42 ns	141 ns	305 ns
SpawnEntity (with string)	12.5 ns	26 ns	58 ns	143 ns	282 ns
Snapshot (32 entities)	10.8 ns	36.7 ns	491 ns	1,997 ns	8,265 ns
Snapshot (1024 entities)	238 ns	335 ns	14,249 ns	63,875 ns	254,741 ns

Only MemoryPack stays in the same order of magnitude, and it still trails by 1.4 to 3.6×: the **fastest zero-allocation encode in .NET**. The honest caveat: Zao isn't the smallest on the wire. MessagePack's varint packing beats it on small, integer-heavy messages. Details: drylane.io/zao.html.

Relay vs NATS: pub/sub

SCENARIO	RELAY (EMBEDDED)	NATS
Live pub to sub, sustained	~6 M msg/s	~1.3 M msg/s (core)
Durable pub to sub, persisted	~4 M msg/s	~0.15 M msg/s (JetStream)
Publish to handler, p50	1–3 µs	~11 µs
Request/reply round trip, p50	3–7 µs	~20 µs
Replay persisted history	~50 M msg/s	~1.3 M msg/s (JetStream)

Replay is where the fused design shows most: catching up is a column scan, not a stream of per-message round trips. Details: drylane.io/relay.html.

On methodology. Rivals got their best configuration: all cores, fastest bulk load, parallel connections, reused buffers. The places the Dry Stack loses are shown in the same tables as the places it wins. The live benchmark also duty-cycles its ingest on purpose: run flat out around the clock it would wear through a demo SSD in days, so it's capped to spare the disk. The speed is real; the cap protects the hardware.

"Single box", is that serious?

The most common objection, and a fair one. Single-box is the *fast path*, and for a surprising share of real workloads it's also the whole answer. But it isn't the only mode.

Facts includes replication with **veto-based failover**: a follower that wants to take over asks first, and a leader that is still alive vetoes it, so two leaders can never exist. The loss window is bounded and well defined: on a hard leader crash, the segment it was still filling. Data is written in immutable, self-

describing segments, each checksummed and recoverable on its own, so a single bad block never takes the dataset down and power loss doesn't corrupt the store. You can break a two-node cluster on purpose and watch it heal, live, at chaos.drylane.io.

For scale-out, a dataset splits into independent **shards** for parallel ingest and query: throughput scales with cores, with no cross-shard coordination on the hot path. The goal is to make one machine embarrassingly capable first, so that "add a node" is a real decision rather than a reflex.

Comfortable to build on

A fast engine you can't stand to use is a benchmark, not a foundation. The application layer is C#, chosen on purpose: it's productive, it's memory-safe, and it keeps getting faster **for free**. Microsoft has spent years optimizing the .NET runtime and GC, and every release hands that work to the code on top. The C++ core holds the floor on raw speed; you write ordinary C#, and the zero-copy wrappers make sure the hot path never pays for the convenience.

Setup is code, not configuration: open a database in a directory, describe your shards, done. No broker, no cluster manager, no YAML. Every setting with its default, and the honest limits, are at drylane.io/setup.html. AI coding assistants get a plain-text summary at drylane.io/llms.txt.

What's next (the TODO list)

None of the below is shipped. It's the roadmap, written as what it is: a list of things not done yet. The through-line is the same: take the optimization lens the stack already proves out, and point it at the next domain where "good enough" has quietly become the ceiling.

- ☐ **A new take on models that never stop learning:** a design in progress, built on foundations that already ingest and replay live data at these speeds.
- ☐ **Deployment via a launcher and updater:** ship and self-update these workloads the way games do, not the container-and-cluster default.
- ☐ **Audio processing:** real-time DSP on the same foundations.
- ☐ **Image processing:** GPU-accelerated (NVIDIA CUDA) pipelines feeding the same storage and transport.
- ☐ **General release:** packaging the stack for everyone. Until then, early access.

These are directions, not promises. They're here because the thesis generalizes: most of the overhead we accept as fixed isn't.

Built by Drylane

Drylane builds the Dry Stack on 40+ years of low-level systems work, from real-time audio DSP to two decades of embedded payment and industrial systems. The closest relative is a from-scratch time-series database that is now the standard for vessel automation and event data at a leading player in

the marine industry. The Dry Stack takes the same ideas further: fused, fast, and honest about its trade-offs.

Early access. We're taking a few design partners. You get the stack and direct engineering help; we get your feedback and, if it works for you, a reference. Production licences are per deployment, simple and fair. Get in touch: info@drylane.io.

See it all live: drylane.io, [market wall](#), [F1 replay](#), [replication and failover](#), [benchmarks](#), [Raspberry Pi](#)

Everything in this document can be checked live at drylane.io. Figures are medians on an AMD Ryzen 9 9950X; each section links the page with the details. Share it, print it, feed it to your favourite model and interrogate it. The numbers hold up to scrutiny, which is the whole idea.